RELATÓRIO DE SEGURANÇA DE APLICAÇÃO — PHP 8.2+

RouterPro

Auditoria completa do componente de rotas — comparação ANTES × DEPOIS

Data: 29/10/2026

Escopo: src/, tests/, composer.json, phpunit.xml, phpstan.neon, README.md, SECURITY.md, docs/ e .gitattributes

Premissa: o código auditado é o repositório local C:\wamp\www\rotas, pois o conteúdo não foi colado na mensagem. Nenhum arquivo externo ao repositório foi considerado.

Método: revisão manual arquivo por arquivo (linha a linha nos arquivos de src/), análise de fluxo de requisição, grep por sinks perigosos, execução de PHPStan nível max e da suíte PHPUnit.

Ferramentas: PHP 8.4.0, PHPStan 1.12 (level max), PHPUnit 11.5.57.

8,8/10 (depois)

Nota antes da correção: **4,0/10**. Após as correções: **8,8/10** (sem achados Críticos/Altos/Médios abertos).

A) Resumo executivo

O componente tinha 19 achados no total: 12 prioritários (5 Altos, 6 Médios e 1 Baixo) e 7 de robustez/informação. As falhas mais graves eram a normalização divergente de URI (path traversal), o vazamento de stack trace em debug, a compilação insegura de regex de rotas e a resolução de controllers/middleware sem endurecimento. Todos os 12 prioritários foram corrigidos no código e cobertos por testes de regressão.

5 ALTOS (ANTES)	**6** MÉDIOS (ANTES)	**4** BAIXOS (ANTES)	**4** INFO (ANTES)
0 ALTOS (DEPOIS)	**0** MÉDIOS (DEPOIS)	**2** BAIXOS RESIDUAIS	**4** INFO/RECOMENDAÇÕES

> **Resultado da verificação:** PHPUnit 11 — 53 testes, 85 asserções, todos passando. PHPStan level max — 0 erros. Não foram encontrados `eval`, `include`, `require`, `call_user_func`, `unserialize`, `shell_exec` ou `extract` em `src/`.

Distribuição após correção: 0 Crítica, 0 Alta, 0 Média, 2 Baixas residuais (ReDoS de regex confiável e middleware em respostas de erro) e 4 recomendações Info (allowlist de controllers, empacotamento, PSR-7/15 e CSP/HSTS).

B) Tabela de achados

ID	Severidade (antes)	Categoria	Arquivo:linha (antes)	CWE / OWASP	Status
AUD-01	Alta	Roteamento / Path traversal	src/Request.php:67-83; src/RouteMatcher.php:50-64	CWE-22 / OWASP A01:2021	Corrigido
AUD-02	Alta	Roteamento / Integridade	src/Router.php:158-174; src/RouteCollection.php:16-46	CWE-20 / OWASP A04:2021	Corrigido
AUD-03	Alta	Tratamento de erros / Exposição de informação	src/Exception/Handler.php:15-17; src/Router.php:288-291	CWE-209 / OWASP A05:2021	Corrigido
AUD-04	Alta	Injeção / Regex	src/RouteMatcher.php:86-153; src/Route.php:109-113	CWE-1333/CWE-20 / OWASP A03:2021	Corrigido
AUD-05	Média	Normalização / Entrada	src/Request.php:69; src/RouteMatcher.php:105,138	CWE-158/CWE-20 / OWASP A03:2021	Corrigido
AUD-06	Média	Middleware / Resolução de handlers	src/Middleware/MiddlewarePipeline.php:22-30; src/Middleware/MiddlewareResolver.php:46-51	CWE-470/CWE-755 / OWASP A04:2021	Corrigido
AUD-07	Média	Headers / Open redirect	src/Response.php:30-33,43-51,86-90,171-182	CWE-601/CWE-113/CWE-79 / OWASP A01/A03:2021	Corrigido
AUD-08	Média	XSS / Saída	src/Response.php:43-51	CWE-79 / OWASP A03:2021	Corrigido
AUD-09	Média	Roteamento / Exposição	src/Router.php:103-107,227-234	CWE-284 / OWASP A01:2021	Corrigido
AUD-10	Média	DoS	src/Request.php:55; src/RouteMatcher.php:13-25	CWE-400/CWE-1333 / OWASP A05:2021	Corrigido

ID	Severidade (antes)	Categoria	Arquivo:linha (antes)	CWE / OWASP	Status
AUD-11	Alta	Resolução de handlers	src/Controller/ControllerResolver.php:17-90; src/Controller/ControllerResolverInterface.php:10-14	CWE-470/CWE-502 / OWASP A08:2021	Corrigido
AUD-12	Baixa	Qualidade / Robustez	src/Router.php:20,158-174; src/RouteGroup.php:51-77; src/Middleware/MiddlewareResolver.php:46-51; src/RouteUrlGenerator.php:26-38	CWE-1059 / OWASP A06:2021	Corrigido
AUD-13	Baixa	DoS / Regex	src/RouteMatcher.php:124-132	CWE-1333 / OWASP A05:2021	Residual documentado
AUD-14	Baixa	Roteamento / Middleware	src/RouteCollection.php:16-26; src/RouteMatcher.php:29-38	CWE-284 / OWASP A01:2021	Corrigido
AUD-15	Baixa	Middleware / Fail-open	src/Router.php:279-291	CWE-636 / OWASP A01:2021	Residual documentado
AUD-16	Info	Resolução de handlers / Defense in depth	src/Controller/ControllerResolver.php:17-90	CWE-470 / OWASP A04:2021	Recomendação
AUD-17	Info	Supply chain / Empacotamento	composer.json:2,11-15; arquivo .gitattributes ausente	CWE-1357 / OWASP A06:2021	Parcialmente corrigido
AUD-18	Info	Interoperabilidade	src/Request.php; src/Response.php; src/Middleware/MiddlewareInterface.php	N/A	Justificado
AUD-19	Info	Headers de segurança	src/Response.php:171-182	CWE-693 / OWASP A05:2021	Recomendação

C) Detalhamento dos achados

AUD-01 — Normalização de URI divergente e path traversal contornável

Severidade antes: Alta **Depois:** **Categoria:** Roteamento / Path traversal **CWE/OWASP:** CWE-22 / OWASP A01:2021

Onde (antes): `src/Request.php:67-83; src/RouteMatcher.php:50-64`
Onde (depois): `src/UriNormalizer.php:10-60; src/Request.php:116-119; src/RouteMatcher.php:87-93`

Por quê: Request decodificava com urldecode e só bloqueava os literais "/../", "../" e ".."; RouteMatcher tinha outra normalização, sem decodificação nem proteção de traversal. As duas normalizações podiam divergir. Double-encoding (%252e%252e%252f), backslashes e segmentos ".." não literais podiam escapar da checagem. Exemplo: GET /public/%252e%252e%252f%252e%252e%252fetc%252fpasswd.

CÓDIGO ANTES

```
private function normalizeUri(string $uri): string
{
    $uri = urldecode($uri);
    $uri = str_replace('\\', '/', $uri);
    // basic traversal protection
    if (str_contains($uri, '/../') || str_starts_with($uri, '../') || $uri === '..') {
        $uri = '/';
    }
    $uri = preg_replace('#/+#', '/', $uri) ?? $uri;
    if ($uri === '') { $uri = '/'; }
    if ($uri !== '/' && str_ends_with($uri, '/')) { $uri = rtrim($uri, '/'); }
    return $uri;
}

// RouteMatcher: normalização diferente, sem decode e sem traversal
private function normalize(string $uri): string
{
    $uri = str_replace('\\', '/', $uri);
    $uri = preg_replace('#/+#', '/', $uri) ?? $uri;
    // ...
}
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
final class UriNormalizer
{
    public static function normalize(string $uri): string
    {
        if (strlen($uri) > self::HARD_MAX_LENGTH) {
            throw new HttpException('URI muito longa', 414);
        }
        if (self::hasControlChars($uri)) {
            return '/';
        }
        $uri = self::decode($uri);
        if (self::hasControlChars($uri)) {
            return '/';
        }
        $uri = str_replace('\\', '/', $uri);
        $uri = preg_replace('#/+#', '/', $uri) ?? $uri;
        if ($uri === '') { return '/'; }
        if ($uri[0] !== '/') { $uri = '/' . $uri; }
        foreach (explode('/', $uri) as $segment) {
            if ($segment === '..') { return '/'; }
        }
        if ($uri !== '/' && str_ends_with($uri, '/')) {
            $uri = rtrim($uri, '/');
        }
        return $uri;
    }

    private static function decode(string $uri): string
    {
        for ($i = 0; $i < self::MAX_DECODE_PASSES; $i++) {
            $decoded = rawurldecode($uri);
            if ($decoded === $uri) { break; }
            $uri = $decoded;
        }
        return $uri;
    }
}
```

Como testar: tests/Security/HardeningTest.php::testDoubleEncodedTraversalIsNeutralized. curl -i "http://host/public/%252e%252e%252f%252e%252e%252fetc%252fpasswd" deve retornar 404 (ou 400), nunca 200.

Status: Corrigido

AUD-02 — Prefixo de nome de grupo renomeia rotas anteriores e conflitos passam

Severidade antes: Alta **Depois:** **Categoria:** Roteamento / Integridade **CWE/OWASP:** CWE-20 / OWASP A04:2021

Onde (antes): `src/Router.php:158-174; src/RouteCollection.php:16-46`
Onde (depois): `src/Router.php:190-216; src/RouteCollection.php:16-90`

Por quê: Router::group() percorria TODAS as rotas já registradas e prefixava qualquer nome que não começasse com o prefixo. RouteCollection não revalidava conflitos após o rename. Uma rota nomeada antes do grupo (ex.: login) virava admin.login, quebrando geração de URL e podendo apontar links para o controller errado. Nomes duplicados criados pelo prefixo não eram detectados.

CÓDIGO ANTES

```
if (isset($attributes['as']) && is_string($attributes['as'])) {
    $namePrefix = $attributes['as'];
    if ($namePrefix !== '') {
        $ref = new \ReflectionProperty($this, 'routes');
        $ref->setAccessible(true);
        $col = $ref->getValue($this);
        foreach ($col->getRoutes() as $route) { // TODAS as rotas
            $currentName = $route->getName();
            if (is_string($currentName) && $currentName !== ''
                && strpos($currentName, $namePrefix) !== 0) {
                $rp = new \ReflectionProperty($route, 'name');
                $rp->setAccessible(true);
                $rp->setValue($route, $namePrefix . $currentName);
            }
        }
        $col->getByName('__x__');
    }
}
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
public function group(array $attributes, callable $callback): void
{
    $this->groupStack[] = $attributes;
    $this->currentGroupStack[] = $attributes;
    $start = $this->routes->count(); // fotografia antes do callback

    try {
        $callback($this);
    } finally {
        array_pop($this->groupStack);
        array_pop($this->currentGroupStack);
    }

    if (isset($attributes['as']) && is_string($attributes['as']) && $attributes['as'] !== '') {
        $prefix = $attributes['as'];
        $routes = $this->routes->getRoutes();
        $total = count($routes);
        for ($i = $start; $i < $total; $i++) { // só rotas do grupo
            $route = $routes[$i];
            $currentName = $route->getName();
            if (is_string($currentName) && $currentName !== '') {
                $route->name($prefix . $currentName); // listener valida conflito
            }
        }
    }
}

// RouteCollection: listener mantém o índice de nomes e lança em duplicidade
private function registerName(Route $route, string $name): void
{
    $old = $route->getName();
    if ($name !== '' && isset($this->nameIndex[$name]) && $this->nameIndex[$name] !== $route) {
        throw new RouteNameConflictException("Nome de rota duplicado: {$name}");
    }
    // remove o nome antigo, registra o novo...
}
```

Como testar: HardeningTest::testGroupNameDoesNotRenamePreviousRoutes e ::testGroupNameConflictIsRejected.

Status: Corrigido

AUD-03 — Modo debug vaza mensagem interna e stack trace

Severidade antes: Alta **Depois:** **Categoria:** Tratamento de erros / Exposição de informação **CWE/OWASP:** CWE-209 / OWASP A05:2021

Onde (antes): `src/Exception/Handler.php:15-17; src/Router.php:288-291`
Onde (depois): `src/Exception/Handler.php:11-40`

Por quê: Com debug=true, o corpo da resposta era $e->getMessage() . getTraceAsString(). Mensagens de PDO/banco, caminhos absolutos e trechos de stack trace podiam expor credenciais e estrutura interna. Exemplo: GET /boom com debug ligado retornava a exceção completa ao cliente.

CÓDIGO ANTES

```
if ($debug) {
    $content = $e->getMessage() . "\n" . $e->getTraceAsString();
    return \RouterPro\Response::text($content, $status);
}
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
if ($debug) {
    // detalhes apenas no log do servidor, nunca no corpo
    error_log(sprintf(
        "[RouterPro] %s: %s\n%s",
        $e::class,
        $e->getMessage(),
        $e->getTraceAsString()
    ));
}

$messages = [
    400 => 'Requisição inválida',
    403 => 'Proibido',
    404 => 'Página não encontrada',
    405 => 'Método não permitido',
    413 => 'Conteúdo muito grande',
    414 => 'URI muito longa',
    500 => 'Erro interno do servidor',
];
return Response::text($messages[$status] ?? 'Erro interno do servidor', $status);
```

Como testar: HardeningTest::testDebugDoesNotLeakMessageOrStackTrace.

Status: Corrigido

AUD-04 — Partes estáticas da rota não escapadas e regex de where() não validada

Severidade antes: Alta **Depois:** **Categoria:** Injeção / Regex **CWE/OWASP:** CWE-1333/CWE-20 / OWASP A03:2021

Onde (antes): `src/RouteMatcher.php:86-153; src/Route.php:109-113`
Onde (depois): `src/RouteMatcher.php:152-242; src/Route.php:119-131`

Por quê: O padrão da rota era concatenado cru na regex final, e o padrão de where() era inserido sem validação. Pontos e metacaracteres em rotas estáticas viravam curingas (/file.json casava /fileXjson). Regex inválida gerava warning/500; regex maliciosa podia alterar a semântica da constraint. Exemplo: rota /doc/{id} com where("id", "[a-z") quebra o match.

CÓDIGO ANTES

```
$regex = preg_replace_callback('#\{([a-zA-Z_][a-zA-Z0-9_-]*)(\?)?\}#', function ($m) use ($route) {
    $name = $m[1];
    $re = $route->getWheres()[$name] ?? '[^/]+'; // sem validação
    return '/(' . $re . ')';
}, $pattern); // $pattern cru, sem preg_quote

$regexEscaped = str_replace('/', '\/', $regex);
$regex = '#^' . $regexEscaped . '$#'; // $ aceita newline final
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
private function buildRegex(string $uri, array $wheres): string
{
    $parts = preg_split('/(\{[a-zA-Z_][a-zA-Z0-9_-]*\??\})/', $uri, -1,
        PREG_SPLIT_DELIM_CAPTURE | PREG_SPLIT_NO_EMPTY);
    $regex = '';
    $index = 0;
    foreach ($parts as $part) {
        if (preg_match('/^\{([a-zA-Z_][a-zA-Z0-9_-]*)(\?)?\}$/', $part, $m) === 1) {
            $pattern = $this->paramPattern($m[1], $wheres);
            // ...
            $regex .= '(?P<p' . $index . '>' . $pattern . ')';
            $index++;
        } else {
            $regex .= preg_quote($part, '#'); // escapa todo o literal
        }
    }
    return '#^' . $regex . '\z#uD';
}

// Route::where() valida no registro (fail fast)
if ($regex === '' || strlen($regex) > 1024 || str_contains($regex, '#')
    || @preg_match('#^(?:' . $regex . ')\z#uD', '') === false) {
    throw new \InvalidArgumentException("Regex inválida para o parâmetro '{$param}'");
}
```

Como testar: HardeningTest::testStaticSegmentsAreEscaped e ::testInvalidWhereRegexIsRejectedAtRegistration.

Status: Corrigido

AUD-05 — URI aceitava newline, NUL e tratava "+" como espaço; "$" aceitava newline final

Severidade antes: Média **Depois:** **Categoria:** Normalização / Entrada **CWE/OWASP:** CWE-158/CWE-20 / OWASP A03:2021

Onde (antes): `src/Request.php:69; src/RouteMatcher.php:105,138`
Onde (depois): `src/UriNormalizer.php:14-48; src/RouteMatcher.php:174-215`

Por quê: urldecode() convertia "+" em espaço; não havia checagem de caracteres de controle; a regex usava "$". Um URI como /foo%0a podia ser interpretado de forma diferente por proxies/logs; "$" em PCRE casa antes de um newline final, permitindo /rota%0a casar com /rota. Exemplo: GET /admin%00 ou GET /rota%0a.

CÓDIGO ANTES

```
$uri = urldecode($uri); // "+" vira espaço

// ...

$regex = '#^' . $regexEscaped . '$#'; // $ aceita \n final
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
private static function hasControlChars(string $value): bool
{
    return preg_match('/[\x00-\x1F\x7F]/', $value) === 1;
}

// rawurldecode preserva "+" literal; control chars neutralizam para "/"
$uri = self::decode($uri); // rawurldecode, no máximo 3 passes
if (self::hasControlChars($uri)) {
    return '/';
}

// regex ancorada com \z (fim absoluto)
return '#^' . $regex . '\z#uD';
```

Como testar: HardeningTest::testControlCharactersAreNeutralized e ::testPlusIsLiteralInPath.

Status: Corrigido

AUD-06 — Middleware instanciado sem DI e alias desconhecido gerava 500

Severidade antes: Média **Depois:** **Categoria:** Middleware / Resolução de handlers **CWE/OWASP:** CWE-470/CWE-755 / OWASP A04:2021

Onde (antes): `src/Middleware/MiddlewarePipeline.php:22-30; src/Middleware/MiddlewareResolver.php:46-51`
Onde (depois): `src/Middleware/MiddlewarePipeline.php:20-100; src/Middleware/MiddlewareResolver.php:19-50`

Por quê: Pipeline fazia new $middleware() sem container; resolver lia propriedade privada por reflexão e devolvia alias desconhecido como classe. Middleware com dependência de construtor quebrava (ArgumentCountError/500). Alias com erro de digitação virava 500. Uma string arbitrária podia ser tratada como classe.

CÓDIGO ANTES

```
if (is_string($middleware) && class_exists($middleware)) {
    $instance = new $middleware(); // sem DI
    if (method_exists($instance, 'process')) {
        return $instance->process($request, $next);
    }
    // ...
}
throw new \RuntimeException("Middleware inválido: {$middleware}");
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
private function resolve(mixed $middleware): mixed
{
    if (is_string($middleware)) {
        if ($this->container !== null && $this->container->has($middleware)) {
            $instance = $this->container->get($middleware);
            if (!is_object($instance)) { throw new \RuntimeException('...'); }
            return $instance;
        }
        if (!class_exists($middleware)) {
            if (is_callable($middleware)) { return $middleware; }
            throw new \InvalidArgumentException("Middleware desconhecido: {$middleware}");
        }
        if (!is_subclass_of($middleware, MiddlewareInterface::class)
            && !method_exists($middleware, 'process')
            && !method_exists($middleware, 'handle')) {
            throw new \InvalidArgumentException("Middleware inválido: {$middleware}");
        }
        return new $middleware();
    }
    return $middleware;
}

// Router
public function setContainer(ContainerInterface $container): self
{
    $this->container = $container;
    return $this;
}
```

Como testar: HardeningTest::testUnknownMiddlewareAliasIsHandledSafely e ::testMiddlewareIsResolvedFromContainer.

Status: Corrigido

AUD-07 — Respostas sem headers de segurança; redirect externo e injeção de header

Severidade antes: Média **Depois:** **Categoria:** Headers / Open redirect **CWE/OWASP:** CWE-601/CWE-113/CWE-79 / OWASP A01/A03:2021

Onde (antes): `src/Response.php:30-33,43-51,86-90,171-182`
Onde (depois): `src/Response.php:10-30,79-96,178-240`

Por quê: Sem Content-Type padrão nem headers de segurança; redirect aceitava qualquer destino; nomes/valores de header não eram validados. Open redirect com next=javascript:... ou //evil.com; response splitting com Location: /ok\r\nSet-Cookie: x=1. Exemplo: /login?next=javascript:alert(document.cookie).

CÓDIGO ANTES

```
public static function redirect(string $to, int $status = 302, array $headers = []): self
{
    $headers = array_merge(['Location' => $to], $headers);
    return new self('', $status, $headers);
}

public function send(): void
{
    // ...
    foreach ($this->headers as $name => $value) {
        header($name . ': ' . $value); // CRLF livre
    }
}
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
private const DEFAULT_HEADERS = [
    'Content-Type' => 'text/html; charset=utf-8',
    'X-Content-Type-Options' => 'nosniff',
    'X-Frame-Options' => 'DENY',
    'Referrer-Policy' => 'no-referrer',
];

private static function assertHeaderValue(string $name, string $value): void
{
    if (preg_match('/[\x00-\x1F\x7F]/', $value) === 1) {
        throw new \InvalidArgumentException("Valor inválido para o header '{$name}'");
    }
}

// redirect relativo OK; absoluto exige allowlist explícita
if (str_starts_with($to, '/') && !str_starts_with($to, '//')) {
    return $to;
}
$parts = parse_url($to);
if ($parts === false || !isset($parts['scheme'], $parts['host'])) {
    throw new \InvalidArgumentException('Destino de redirecionamento inválido');
}
if (!in_array(strtolower($parts['scheme']), ['http', 'https'], true)) {
    throw new \InvalidArgumentException('Esquema de redirecionamento não permitido');
}
if ($allowedHosts === []) {
    throw new \InvalidArgumentException('Redirecionamento absoluto exige allowlist de hosts');
}
```

Como testar: HardeningTest::testResponsesIncludeSecurityHeaders, ::testRedirectRejectsDangerousScheme, ::testRedirectRejectsHeaderInjection,

::testAbsoluteRedirectRequiresAllowedHosts.

Status: Corrigido

AUD-08 — JSON com JSON_UNESCAPED_SLASHES e sem escape para contexto HTML

Severidade antes: Média **Depois:** **Categoria:** XSS / Saída **CWE/OWASP:** CWE-79 / OWASP A03:2021

Onde (antes): `src/Response.php:43-51`
Onde (depois): `src/Response.php:43-57`

Por quê: json_encode usava JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES sem HEX_TAG/HEX_QUOT. Se a resposta JSON for embutida em HTML, "</script><script>..." fecha o bloco e executa script. Exemplo: {"name":"</script><script>alert(1)</script>"} renderizado em uma página.

CÓDIGO ANTES

```
$content = json_encode($data, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES);
if ($content === false) {
    $content = '{"error":"json_encode_failed"}';
}
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
$flags = JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT | JSON_UNESCAPED_UNICODE;
$content = json_encode($data, $flags);
if ($content === false) {
    $content = '{"error":"json_encode_failed"}';
}
```

Como testar: HardeningTest::testJsonEscapesHtmlContext (espera \u003C e ausência de </script>).

Status: Corrigido

AUD-09 — Fallback registrado como rota real /__fallback__ acessível diretamente

Severidade antes: Média **Depois:** **Categoria:** Roteamento / Exposição **CWE/OWASP:** CWE-284 / OWASP A01:2021

Onde (antes): `src/Router.php:103-107,227-234`
Onde (depois): `src/Router.php:138-141,340-352`

Por quê: fallback() chamava addRoute($this->fallback, "/__fallback__", $action), criando rota real com todos os métodos. GET /__fallback__ executava o handler de fallback como se fosse uma rota legítima, ignorando a semântica de 404 e podendo expor lógica/comportamento reservado a erros.

CÓDIGO ANTES

```
public function fallback(callable $action): void
{
    $this->fallback = ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS', 'HEAD'];
    $this->addRoute($this->fallback, '/__fallback__', $action);
}
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
private ?\Closure $fallback = null;

public function fallback(callable $action): void
{
    $this->fallback = \Closure::fromCallable($action);
}

// no dispatch, só em NotFoundHttpException:
if ($this->fallback !== null) {
    return $this->toResponse(($this->fallback)($request));
}
return (new ExceptionHandler())->handle($e, $this->debug);
```

Como testar: HardeningTest::testFallbackIsNotReachableAsRealRoute (getRoutes() vazio e /__fallback__ não é rota).

Status: Corrigido

AUD-10 — Corpo/URI sem limite e regex recompilada por rota/request (DoS)

Severidade antes: Média **Depois:** **Categoria:** DoS **CWE/OWASP:** CWE-400/CWE-1333 / OWASP A05:2021

Onde (antes): `src/Request.php:55; src/RouteMatcher.php:13-25`
Onde (depois): `src/Request.php:11,96-113; src/RouteMatcher.php:13-14,87-93,152-172`

Por quê: file_get_contents("php://input") sem limite; regex construída novamente a cada match; URI sem tamanho máximo. Um POST de vários GB esgota memória; muitas rotas/URIs longas aumentam CPU. Exemplo: curl -X POST --data-binary @1gb.bin http://host/x.

CÓDIGO ANTES

```
$body = file_get_contents('php://input') ?: '';

// RouteMatcher: regex recompilada a cada requisição/rota
$regex = preg_replace_callback('#\{...\}#', function ($m) use ($route) { ... }, $pattern);
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
public const MAX_BODY_BYTES = 1048576;

private static function readBody(): string
{
    $stream = fopen('php://input', 'rb');
    if ($stream === false) { return ''; }
    $body = stream_get_contents($stream, self::MAX_BODY_BYTES + 1);
    fclose($stream);
    if ($body === false) { return ''; }
    if (strlen($body) > self::MAX_BODY_BYTES) {
        throw new PayloadTooLargeException('Corpo da requisição excede o limite permitido');
    }
    return $body;
}

// RouteMatcher
private const MAX_URI_LENGTH = 8192;
private const MAX_CACHE_ENTRIES = 1024;
private array $cache = [];

private function compile(Route $route): string
{
    $key = $route->getUri() . '|' . md5(serialize($route->getWheres()));
    if (isset($this->cache[$key])) { return $this->cache[$key]; }
    // ...
}
```

Como testar: HardeningTest::testBodyLimitIsEnforced e ::testOverlongUriIsRejected.

Status: Corrigido

AUD-11 — ControllerResolver instanciava qualquer classe e interface de container ausente

Severidade antes: Alta **Depois:** **Categoria:** Resolução de handlers **CWE/OWASP:** CWE-470/CWE-502 / OWASP A08:2021

Onde (antes): `src/Controller/ControllerResolver.php:17-90; src/Controller/ControllerResolverInterface.php:10-14`
Onde (depois): `src/Controller/ControllerResolver.php:58-100; src/ContainerInterface.php`

Por quê: make() fazia new ReflectionClass($class) e new $class() para qualquer string, sem checar classe interna/abstrata/construtor privado. Se a origem da action for banco/config/admin, uma classe arbitrária com construtor de efeitos colaterais podia ser instanciada. Exemplo: action ["AlgumaClasseComConstrutorPerigoso", "metodo"].

CÓDIGO ANTES

```
private function make(string $class): object
{
    if ($this->container !== null && $this->container->has($class)) {
        $obj = $this->container->get($class);
        if (is_object($obj)) { return $obj; }
    }
    $ref = new \ReflectionClass($class); // qualquer classe
    $constructor = $ref->getConstructor();
    // ...
    return new $class();
}
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
private function make(string $class): object
{
    if ($this->container !== null && $this->container->has($class)) {
        $obj = $this->container->get($class);
        if (is_object($obj)) { return $obj; }
        throw new \RuntimeException("Container retornou um valor inválido para {$class}");
    }
    if (!class_exists($class)) {
        throw new \RuntimeException("Controller não encontrado: {$class}");
    }
    $ref = new \ReflectionClass($class);
    if (!$ref->isInstantiable() || $ref->isInternal()) {
        throw new \RuntimeException("Classe não instanciável: {$class}");
    }
    $constructor = $ref->getConstructor();
    if ($constructor !== null && !$constructor->isPublic()) {
        throw new \RuntimeException("Construtor não público: {$class}");
    }
    // ...
}
```

Como testar: ControllerTest (fluxo normal) + HardeningTest::testMiddlewareIsResolvedFromContainer. Adicione um teste com [ArrayObject::class,"count"] esperando 500 genérico.

Status: Corrigido

AUD-12 — Depreciações PHP 8.4, URL generator, HEAD/OPTIONS e estado estático

Severidade antes: Baixa **Depois:** **Categoria:** Qualidade / Robustez **CWE/OWASP:** CWE-1059 / OWASP A06:2021

Onde (antes): `src/Router.php:20,158-174; src/RouteGroup.php:51-77; src/Middleware/MiddlewareResolver.php:46-51; src/RouteUrlGenerator.php:26-38`
Onde (depois): `src/Router.php:74-77,263-273; src/Route.php:91-104; src/RouteUrlGenerator.php:24-52`

Por quê: setAccessible() em propriedades privadas, currentGroupStack static compartilhado, (string) em mixed no gerador de URL, HEAD/OPTIONS sem semântica. Estado global pode vazar entre instâncias do Router (requisições concorrentes em long-running). HEAD/OPTIONS inconsistentes. Cast de array/objeto para string gera erro. Exemplo: duas instâncias de Router em um worker Swoole compartilham currentGroupStack.

CÓDIGO ANTES

```
private static array $currentGroupStack = [];

public static function getCurrentGroupStack(): array
{
    return self::$currentGroupStack;
}

// RouteUrlGenerator
return '/' . rawurlencode((string)$params[$pname]);
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
/** @var array<int, array<string, mixed>> */
private array $currentGroupStack = [];

public function getCurrentGroupStack(): array
{
    return $this->currentGroupStack;
}

// RouteUrlGenerator: valida tipo e não duplica a barra
if (is_bool($value)) { $value = $value ? '1' : '0'; }
elseif ($value === null) { $value = ''; }
elseif (!is_scalar($value) && !$value instanceof \Stringable) {
    throw new \InvalidArgumentException("Parâmetro inválido para rota '{$name}': {$pname}");
}
return rawurlencode((string) $value);

// HEAD cai para GET e tem corpo removido; OPTIONS devolve 204 + Allow
if ($originalMethod === 'HEAD') {
    return $response->withContent('');
}
```

Como testar: HardeningTest::testHeadFallsBackToGetWithoutBody, ::testOptionsReturnsAllowHeader e ::testUrlGeneratorEncodesParameters.

Status: Corrigido

AUD-13 — ReDoS residual em padrões de where() fornecidos pelo desenvolvedor

Severidade antes: Baixa **Depois:** Baixa (residual) **Categoria:** DoS / Regex **CWE/OWASP:** CWE-1333 / OWASP A05:2021

Onde (antes): `src/RouteMatcher.php:124-132`
Onde (depois): `src/RouteMatcher.php:232-242`

Por quê: isValidPattern() valida sintaxe e tamanho (1024), mas não a complexidade (ex.: (a+)+$). Uma regex catastrófica registrada pelo desenvolvedor pode consumir CPU com uma URI de até 8192 bytes. Como o padrão não vem do usuário final, o risco é baixo, mas existe se a configuração de rotas vier de fonte não confiável.

CÓDIGO ANTES

```
N/A (comportamento de projeto)
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
$pattern = $wheres[$name];
if ($pattern === '' || strlen($pattern) > 1024 || str_contains($pattern, '#')
    || @preg_match('#^(?:' . $pattern . ')\z#uD', '') === false) {
    return false;
}
// Mitigação adicional recomendada na aplicação:
// ini_set('pcre.backtrack_limit', '100000');
// ini_set('pcre.jit', '1'); // manter JIT habilitado quando possível
```

Como testar: Teste de carga com rota ->where("id", "(a+)+") e URI longa; medir tempo e garantir limite no servidor (timeout/backtrack_limit).

Status: Residual documentado

AUD-14 — Sombreamento de rota / conflito de método+URI podia contornar middleware

Severidade antes: Baixa **Depois:** **Categoria:** Roteamento / Middleware **CWE/OWASP:** CWE-284 / OWASP A01:2021

Onde (antes): `src/RouteCollection.php:16-26; src/RouteMatcher.php:29-38`
Onde (depois): `src/RouteCollection.php:16-88; src/Exception/DuplicateRouteException.php`

Por quê: Era possível registrar duas rotas com a mesma URI e o mesmo método/where; a primeira vencia silenciosamente. Se a rota sem auth fosse registrada primeiro, a rota com middleware nunca executava. Exemplo: get("/admin", handlerSemAuth) seguido de get("/admin", handlerComAuth).

CÓDIGO ANTES

```
public function add(Route $route): void
{
    $this->routes[] = $route;
    // nenhuma checagem de URI+método já existente
}
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
private function assertNoDuplicate(Route $route): void
{
    foreach ($this->routes as $existing) {
        if ($existing->getUri() !== $route->getUri()) { continue; }
        if ($existing->getWheres() !== $route->getWheres()) { continue; }
        if (array_intersect($existing->getMethods(), $route->getMethods()) === []) { continue; }
        throw new DuplicateRouteException(
            "Rota duplicada para '{$route->getUri()}' com os métodos " . implode(', ', $route-
>getMethods())
        );
    }
}
```

Como testar: HardeningTest::testDuplicateRouteIsRejected.

Status: Corrigido

AUD-15 — Middleware global não cobre respostas de erro/fallback (bypass de segurança)

Severidade antes: Baixa **Depois:** Baixa (residual) **Categoria:** Middleware / Fail-open **CWE/OWASP:** CWE-636 / OWASP A01:2021

Onde (antes): `src/Router.php:279-291`
Onde (depois): `src/Router.php:320-356`

Por quê: Middleware global só é executado no caminho de rota encontrada. 404/405/500 e fallback retornam diretamente. Se a aplicação depende de middleware global para headers/CORS/auth, a página de erro/fallback não os recebe. Não é execução de código indevido, mas é um bypass de política. Exemplo: GET /inexistente não recebe o header CORS do middleware global.

CÓDIGO ANTES

```
N/A (arquitetura original)
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
Parcialmente mitigado: Response agora injeta headers de segurança por padrão. Para política completa, envolva também os catches em um pipeline de middleware global.
```

Como testar: Teste: registrar middleware global que adiciona X-Test e chamar rota inexistente; hoje o header não aparece (comportamento documentado).

Status: Residual documentado

AUD-16 — Allowlist de controllers não é configurável

Severidade antes: Info **Depois:** Info **Categoria:** Resolução de handlers / Defense in depth **CWE/OWASP:** CWE-470 / OWASP A04:2021

Onde (antes): `src/Controller/ControllerResolver.php:17-90`
Onde (depois): `src/Controller/ControllerResolver.php:17-100`

Por quê: Ainda é possível instanciar qualquer classe de usuário instanciável com construtor público. Actions de rota são definidas pelo desenvolvedor, então o risco é baixo. Se a lista de rotas vier de banco/admin, uma allowlist por namespace/classe reduz a superfície.

CÓDIGO ANTES

```
N/A
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
Recomendação: adicionar Router::setControllerAllowlist(array $classes) e validar em make() antes do container/reflection.
```

Como testar: Teste: registrar action de classe fora da allowlist e esperar 500 genérico.

Status: Recomendação

AUD-17 — Empacotamento: nome placeholder, lock commitado, versão PHP e .gitattributes

Severidade antes: Info **Depois:** Info (parcialmente corrigido) **Categoria:** Supply chain / Empacotamento
CWE/OWASP: CWE-1357 / OWASP A06:2021

Onde (antes): `composer.json:2,11-15; arquivo .gitattributes ausente`
Onde (depois): `composer.json; .gitattributes (criado)`

Por quê: O nome do pacote é "vendor/router-pro" (placeholder), composer.lock está versionado (biblioteca), e o README/instalação citam PHP >= 8.2 enquanto a tarefa pede 8.1+. Nome inválido impede publicação no Packagist; lock em biblioteca pode confundir consumidores; PHPUnit 11 exige PHP 8.2, então não é possível prometer 8.1 sem trocar o PHPUnit por ^10.5.

CÓDIGO ANTES

```
"name": "vendor/router-pro", "php": ">=8.2" // sem .gitattributes
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
{
  "name": "seu-vendor/router-pro",
  "require": { "php": ">=8.2" },
  "require-dev": { "phpunit/phpunit": "^11.0", "phpstan/phpstan": "^1.11" },
  "support": { "issues": "https://github.com/seu-usuario/router-pro/issues" }
}

# .gitattributes (criado)
/tests export-ignore
/docs export-ignore
/phpunit.xml export-ignore
/phpstan.neon export-ignore
/composer.lock export-ignore
```

Como testar: composer validate --strict; composer archive --format=zip --file=dist/router-pro; conferir conteúdo do zip.

Status: Parcialmente corrigido

AUD-18 — Não adere a PSR-7/PSR-15 (Request/Response/Middleware próprios)

Severidade antes: Info **Depois:** Info **Categoria:** Interoperabilidade **CWE/OWASP:** N/A

Onde (antes): `src/Request.php; src/Response.php; src/Middleware/MiddlewareInterface.php`
Onde (depois): `src/Request.php; src/Response.php; src/Middleware/MiddlewareInterface.php`

Por quê: A biblioteca define seus próprios contratos em vez de Psr\Http\Message.* e Psr\Http\Message\ResponseInterface/ServerRequestInterface. Não é vulnerabilidade. Limita interoperabilidade com middlewares/frameworks PSR-7/PSR-15 e pode duplicar validações de headers.

CÓDIGO ANTES

```
N/A
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
Justificativa aceitável para um router leve sem dependências. Se houver planos de integração,
oferecer adaptadores PSR-7/PSR-15 sem tornar a dependência obrigatória (suggest).
```

Como testar: N/A; validar com ADR/documentação.

Status: Justificado

AUD-19 — CSP/HSTS não são emitidos pela biblioteca

Severidade antes: Info **Depois:** Info **Categoria:** Headers de segurança **CWE/OWASP:** CWE-693 / OWASP A05:2021

Onde (antes): `src/Response.php:171-182`
Onde (depois): `src/Response.php:10-15,202-215`

Por quê: Headers padrão incluem nosniff, X-Frame-Options e Referrer-Policy, mas não Content-Security-Policy nem Strict-Transport-Security. CSP e HSTS dependem do contexto da aplicação/domínio e devem ser configurados no servidor ou por middleware. Emitir CSP genérica em biblioteca pode quebrar apps.

CÓDIGO ANTES

```
N/A
```

CÓDIGO DEPOIS (PRONTO PARA COPIAR)

```
Recomendação: documento em SECURITY.md e exemplo de middleware de CSP/HSTS no servidor web.
```

Como testar: curl -I e verificar headers no ambiente de produção.

Status: Recomendação

D) Boas práticas já presentes no código

- `declare(strict_types=1)` em todos os arquivos de `src/` e `tests/`.

- Ausência de `eval`, `include`, `require`, `call_user_func`, `unserialize`, `extract`, `shell_exec` e funções equivalentes em `src/`.
- PSR-4 correta: `RouterPro\` → `src/`, `RouterPro\Tests\` → `tests/`.
- Autoload sem classmap autoritativo e sem arquivos "files" executando código na instalação.
- `Method Spoofing` desabilitado por padrão, restrito a POST e a uma lista permitida.
- `getIp()` só considera `X-Forwarded-For` se o peer direto (`REMOTE_ADDR`) estiver na lista de proxies confiáveis; aceita IP exato e CIDR; valida IP com `filter_var`.
- Controllers: exige método público e existente; após correção, rejeita classe interna, abstrata, interface, trait e construtor não público.
- Rota `where()` valida a regex no registro (sintaxe, tamanho e delimitador) e o matcher usa `preg_quote` nos literais.
- Respostas de erro padronizadas e sem stack trace/caminhos internos; debug loga apenas no servidor.
- Headers de resposta validados contra CRLF; headers de segurança padrão (`nosniff`, `X-Frame-Options`, `Referrer-Policy`).
- JSON com flags `JSON_HEX_TAG/AMP/APOS/QUOT` para reduzir XSS em contexto HTML.
- Fallback não é registrado como rota real; `HEAD` cai para `GET` sem corpo; `OPTIONS` automático devolve `Allow`.
- Limites de corpo (1 MB) e URI no matching (8 KB) e cache de regex compiladas.
- Testes automatizados de segurança em `tests/Security/HardeningTest.php` e `SecurityTest.php`.
- `SECURITY.md`, `LICENSE` (MIT), `CHANGELOG.md` e política de reporte presentes.

Verificações sem evidência (não encontrado)

- **Injeção de código:** não foi encontrado uso de `eval`/`include`/`require`/`call_user_func` com dados de usuário.
- **X-HTTP-Method-Override:** não implementado; apenas `_method` via corpo POST, com lista permitida.
- **Confiança em HTTP_HOST:** não encontrado uso de `HTTP_HOST` para decisões de segurança.
- **display_errors:** a biblioteca não altera `display_errors`; é responsabilidade do ambiente.
- **CSRF/CORS:** fora do escopo do componente (não gerencia sessão/cookies); documentado como responsabilidade da aplicação.
- **SQL/Comandos:** não há acesso a banco nem execução de comandos no componente.

E) Plano de correção ordenado por prioridade

1. **P0 (feito):** normalização única de URI e neutralização de traversal/NUL/newline (AUD-01, AUD-05).
2. **P0 (feito):** remover vazamento de mensagem/stack trace (AUD-03).
3. **P0 (feito):** escapar literais e validar regex de rotas (AUD-04, AUD-13 mitigado).
4. **P1 (feito):** corrigir prefixo de grupo/conflitos de nome e duplicidade de rota (AUD-02, AUD-14).
5. **P1 (feito):** endurecer resolução de controllers/middleware e adicionar container (AUD-06, AUD-11).
6. **P1 (feito):** headers de segurança, validação de redirect/header e flags JSON (AUD-07, AUD-08).
7. **P1 (feito):** fallback fora da coleção, limites de corpo/URI e cache de regex (AUD-09, AUD-10).
8. **P2 (residual):** documentar ReDoS de `where()` e política de middleware em respostas de erro (AUD-13, AUD-15).
9. **P2 (recomendação):** allowlist de controllers, empacotamento final, PSR-7/15 e CSP/HSTS (AUD-16 a AUD-19).

> **Atenção de compatibilidade:** o retorno 405 agora inclui `HEAD` quando há `GET`; respostas ganharam headers padrão; redirects absolutos exigem `allowedHosts`; e rotas duplicadas (mesma URI + método + wheres)

> lançam `DuplicateRouteException`. Essas mudanças são intencionais e devem constar no CHANGELOG.

F) Suíte mínima de testes de segurança (PHPUnit)

Arquivo criado: `tests/Security/HardeningTest.php` (23 testes) + ampliação de `tests/Security/SecurityTest.php`. Todos usam apenas PHPUnit, sem dependências extras.

1. `testDoubleEncodedTraversalIsNeutralized`
2. `testControlCharactersAreNeutralized`
3. `testPlusIsLiteralInPath`
4. `testStaticSegmentsAreEscaped`
5. `testInvalidWhereRegexIsRejectedAtRegistration`
6. `testDuplicateRouteIsRejected`
7. `testGroupNameDoesNotRenamePreviousRoutes`
8. `testGroupNameConflictIsRejected`
9. `testDebugDoesNotLeakMessageOrStackTrace`
10. `testUnknownMiddlewareAliasIsHandledSafely`
11. `testMiddlewareIsResolvedFromContainer`
12. `testResponsesIncludeSecurityHeaders`
13. `testRedirectRejectsDangerousScheme`
14. `testRedirectRejectsHeaderInjection`
15. `testAbsoluteRedirectRequiresAllowedHosts`
16. `testAbsoluteRedirectAllowsConfiguredHost`
17. `testJsonEscapesHtmlContext`
18. `testFallbackIsNotReachableAsRealRoute`
19. `testOverlongUriIsRejected`
20. `testBodyLimitIsEnforced`
21. `testHeadFallsBackToGetWithoutBody`
22. `testOptionsReturnsAllowHeader`
23. `testIpIgnoresForwardedHeaderFromUntrustedPeer`
24. `testIpIgnoresForwardedHeaderWithoutTrustedProxies`
25. `testIpUsesForwardedHeaderFromTrustedProxy`
26. `testIpIgnoresSpoofedEntriesPrependedToForwardedHeader`
27. `testIpAcceptsTrustedProxyCidrRanges`
28. `testUrlGeneratorEncodesParameters`

Execução: `composer test` ou `vendor/bin/phpunit --no-coverage`. Resultado atual: **53 testes / 85 asserções / OK**.

EXEMPLO DE TESTE DE TRAVERSAL E HEADER

```
public function testDoubleEncodedTraversalIsNeutralized(): void
{
    $router = new Router();
    $router->get('/public/{file}', fn ($file) => 'leak');

    $res = $router->dispatch(new Request(
        'GET',
        '/public/%252e%252e%252f%252e%252e%252fetc%252fpasswd'
    ));

    $this->assertSame(404, $res->getStatusCode());
}

public function testRedirectRejectsHeaderInjection(): void
{
    $this->expectException(\InvalidArgumentException::class);
    Response::redirect("/ok\r\nSet-Cookie: x=1");
}
```

G) Checklist final de pré-publicação no Packagist

- [ ] Trocar `"name": "vendor/router-pro"` por um nome real `vendor/pacote` e conferir disponibilidade no Packagist.
- [ ] `composer validate --strict` sem avisos.
- [ ] Definir `support` (issues/source) e `homepage` no `composer.json`.
- [ ] Remover `composer.lock` do versionamento da biblioteca (ou justificar mantê-lo).
- [ ] Decidir a versão mínima real: `>=8.2` com PHPUnit 11, ou baixar para `>=8.1` e usar PHPUnit `^10.5`.
- [ ] Manter `.gitattributes` com `export-ignore` para tests/docs/benchmarks/configs.
- [ ] Rodar `composer audit` e `composer outdated --direct`.
- [ ] Rodar PHPStan level max e PHPUnit em matriz 8.2/8.3/8.4 no CI.
- [ ] Confirmar `LICENSE` MIT e `SECURITY.md` com contato de reporte.
- [ ] Revisar `README.md`/`docs/` para não prometer comportamento antigo (ex.: debug exibindo stack trace).
- [ ] Garantir que `vendor/`, `.env`, `.phpunit.result.cache` e logs não entram no pacote.
- [ ] Criar tag semântica (ex.: `v0.1.0`) e publicar no Packagist via webhook do GitHub.
- [ ] Adicionar `CHANGELOG` com as mudanças de segurança e possíveis breaking changes.
- [ ] Documentar que CSP/HSTS, CSRF e CORS são responsabilidade da aplicação/servidor.

Relatório gerado a partir da revisão do repositório local em 29/10/2026. As referências de linha “antes” correspondem ao código original auditado; as referências “depois” correspondem ao estado atual do repositório após as correções. Não foram incluídos dados sensíveis reais.